Selecting Reliable LLM Reasoning

Question-conditioned verification under a fixed CPU budget

DASE7506 Mini-project 2 | Final model: v14

HE YUXIN | Student ID: 3036897306

Abstract

This project selects a numerically correct solution from four fixed language-model candidates. The proposed system combines a supervised pairwise verifier with an explicit question-conditioned low-rank scorer. It uses only the supplied texts, fixed embeddings and training references. A shared train-only TF-IDF/SVD encoder represents questions and solutions; learned interactions allow the interpretation of a solution feature to depend on the question. Internal training validation selects model settings, and official validation selects the final fusion. The frozen system reaches 388/467 test accuracy (83.08%), compared with 380/467 (81.37%) for the preceding pairwise baseline. A clean standalone reproduction reaches the same score within the CPU and asset limits locally. The gain is small and statistically uncertain; unsuccessful tuning and confidence diagnostics are reported rather than omitted.

1. Task and research question

For each question q, the input comprises four candidate reasoning texts c1,...,c4 and fixed 1,024-dimensional whole-trace embeddings. The output must be one Python integer in {0,1,2,3}. The unchanged official parser extracts a numeric final answer. A selection is correct when that number equals the supplied reference; several candidates may be correct. This objective evaluates answer selection, not whether every reasoning step is valid [1,3].

The central question is whether relative correctness and explicit question-solution interactions provide useful evidence beyond fluent reasoning, answer agreement and independent candidate classification. The study therefore compares a majority baseline, an independent verifier, pairwise learning, mathematical checks, additive text features and learned question conditioning.

Split	Questions	Use
Training	4,755	Weights, vocabulary, SVD and normalization
Validation	798	Development and final fusion selection
Test	467	Frozen scoring and reproduction only
Hard validation	355 of 798	At least two distinct valid numeric answers

Evaluation discipline

All fits use training references only. The final text-model architecture and epoch are chosen on a question-level training-internal holdout; official validation then selects fusion. Earlier baseline test results were already known, and validation was used repeatedly across development. The final v14 weights and configuration were frozen and hashed before its first test scoring. The standalone-folder run is a reproduction of those same assets, not another checkpoint search. IDs serve alignment only. No external training data, additional pretrained weights, online generation or prediction lookup is used.

2. Inputs and the pairwise baseline

2.1 Features and preprocessing

Each supplied embedding is L2-normalized. The original text branch contains 48 scalar features plus 48 within-question centered counterparts: length and formatting, lexical/number overlap with the question, 16 style indicators, numeric-answer properties, answer support/diversity, distance from group median and simple arithmetic checks. Training-only means and standard deviations standardize each coordinate; values are clipped to [-10,10]. A zero-variance coordinate receives scale one.

The extended checker adds 20 features and their 20 centered counterparts. A bounded AST interpreter with exact rational arithmetic supports explicit numeric equalities, parentheses, nested fractions, percentages, limited integer powers, chained equalities and decimal rounding. Unsupported expressions are separated from contradictions. Markdown bullets and percentage displays receive conservative normalization. Final-result mismatch and conflicting answer markers are weak signals, because intermediate quantities and corrected earlier answers can legitimately differ. The final input before text encoding has 1,160 coordinates: 1,024 embedding, 96 original scalar and 40 extended mathematical features.

2.2 Pairwise linear ranking

Let yij indicate whether candidate j for question i has the correct parsed final answer. Form positive-negative pairs only; correct-correct pairs are not arbitrarily ordered. For linear scores sij = w^T xij, minimize a weighted Bradley-Terry logistic loss with L2 regularization [2]. Each question initially contributes total pair weight one; a valid wrong answer receives multiplier three to emphasize substantive disagreements over extraction failures.

```
Lpair = sum_i sum_(j positive,k negative) aijk * softplus(sik - sij).
```

Symmetric feature differences and reversed labels are fitted with scikit-learn LogisticRegression, no intercept, C=.01, max_iter=500 and seed 7506. There are 2,513 mixed-correctness training questions and 8,271 positive-negative pairs. All-correct questions provide no ranking signal, consistent with the group-relative discussion in Lecture 7 [2]. The method is supervised ranking, not generative reinforcement learning.

2.3 Contextual neural scorer and aggregation

A 64-unit ReLU head receives the normalized embedding, 96 scalars and embedding minus the four-candidate mean (2,144 coordinates). Adam uses learning rate .001, batch size 64 questions and L2 coefficient .0001 on weight matrices. The objective is question-normalized pairwise loss plus .2 times binary cross-entropy; the auxiliary term retains signal from all-correct groups. The selected seed is 7506 and the selected checkpoint is epoch 20.

```
baseline_score = .75 * pooled_sigmoid(linear) + .25 * pooled_sigmoid(neural).
```

Pooling averages scores among candidates with the same parsed numeric answer. Invalid answers receive -1, and argmax chooses the first maximum. Mean pooling reduces dependence on which equivalent trace is selected. Sum pooling can amplify correlated errors and was inferior in earlier trials. The resulting v2 baseline reaches 749/798 validation and 380/467 test accuracy. These bounded scores are ranking scores, not automatically calibrated probabilities.

3. Explicit question-conditioned verification

3.1 Shared text encoding and additive control

The earlier lexical branch encoded candidate text alone. The final experiment explicitly encodes questions and solutions separately using a shared representation. Numeric literals are replaced by NUM. Word unigrams/bigrams use sublinear TF-IDF, min_df=3 and a maximum vocabulary of 10,000 terms. TruncatedSVD learns 128 coordinates using five iterations and seed 7506. The fitting corpus contains each training question once and its four solutions. Question and solution vectors are individually L2-normalized. No pretrained text model is imported.

For dimensions 64 and 128, the additive control concatenates existing inputs, solution text vector r and shared question vector q. A linear term depending only on q is identical for all four candidates, so it cancels in pair differences. The implementation asserts that this coefficient vanishes. The fixed-interaction model additionally uses coordinate products q*r, absolute differences |q-r| and cosine similarity. It tests whether explicit text interactions help beyond simply adding lexical information.

3.2 Learnable low-rank interaction

The learned scorer lets question features change the weight assigned to candidate features. Let x concatenate the 1,160-dimensional original/math branch and the 128-dimensional solution text vector. Let q be the 128-dimensional question vector. Both branches use training-only mean/std normalization and clipping.

```
z(q,x) = w^T x + sum_l [(q^T U)_l * (x^T V)_l] / sqrt(r) + b.
```

U and V are trainable low-rank factors. The selected rank is 32, so conditioning is more compact than a full question-by-candidate interaction matrix. The model has 46,601 learned score parameters, excluding standardization and the text encoder. Factor matrices start from seeded small random values and the linear term starts at zero. Adam uses lr=.001, batch=64, linear L2=.0001 and selected factor L2=.01. The objective is question-normalized pairwise loss without the linear baseline's valid-negative multiplier, plus .2 BCE. This trains a task-specific head on fixed representations; it does not fine-tune the generating LLM.

3.3 Frozen final inference

```
final_score = .75 * pooled_sigmoid(z(q,x)) + .25 * baseline_score.
```

All question/solution encoding and feature extraction occur at inference using saved vocabulary, IDF, SVD and standardization arrays. The baseline has its own saved linear and neural weights. Mean pooling handles equal answers, invalid answers remain at -1, and one candidate is returned for every question. The implementation accepts new IDs and arbitrary bank sizes without retraining.

Four synthetic tests verify that an additive question cancels, interactions can change candidate preferences, the bilinear model responds to q, and custom derivatives match finite differences. The maximum derivative error is 9.6e-11. These checks establish numerical implementation correctness, not semantic validity of every learned comparison.

4. Training, tuning and unsuccessful alternatives

4.1 Development stages

Model selection was incremental. The table summarizes actual development, including failed ideas. Historical exploratory code and rejected weights are excluded from the minimal folder; their compact records remain in evidence/. The final selected recipe and the complete v14 search are reproducible with train.py and search.py.

Stage	Change / actual fits	Outcome
v1	Independent linear/tree verifier	Validation 92.36%; test 79.66%
v2	Pairwise linear/neural search; 11 fits	Validation 93.86%; test 81.37%
v3-v4	Answer groups, text, kernels, trees; 46 fits	No strict improvement over v2
v5	Grouped CV, heads, seed refits; 73 fits	Includes 4 low-LR continuations; no gain
v6	Block scaling, margins, hard negatives; 63 fits	Small OOF changes did not transfer
v7-v10	Exact SVM, seeds, hinge, regressions; 48 fits	No strict validation gain
v11	Mathematical features; 38 final / 76 total	Matched control 93.11% to 93.86%
v12-v13	Nonlinear duels and valid-only training; 35 fits	Best new blend 93.73%
v14	Explicit question conditioning; 19 model fits	Selected fusion validation 94.36%

OOF denotes out-of-fold training evaluation. The v11 rerun corrected decimal-percentage rounding; it did not change the parameter grid. Published papers [4,5] motivated verification hypotheses, but their datasets and model weights were not imported.

Head-tuning candidates covered hidden widths 64/128, learning rates .001/.0005, dropout 0/.1, L2 .0001-.001 and BCE weights .1/.2/.5; grouped CV and an internal holdout screened settings before multi-seed refits. Four further continuations of saved v2 weights used learning rates .00003/.0001 with dropout 0/.1 for ten epochs. These continuations reset optimizer moments and did not beat v2. This was head/parameter tuning on fixed embeddings, not LLM weight fine-tuning.

4.2 Final v14 search and fixed settings

A training-only 80/20 split, stratified by number of correct candidates, uses seed 7506: 3,804 internal training and 951 internal holdout questions. All transforms fit internal training only. Holding out whole questions prevents their four related candidates leaking across the split.

Component	Internal comparison	Selected setting
Additive text	d={64,128}; C={.003,.01,.03}	d=128; C=.01
Fixed interactions	Same dimensions and C grid	d=64; C=.003
Learned interaction	rank={16,32}; L2={.001,.01}	rank=32; L2=.01
Learned epoch	Internal checkpoints {10,20,40}	40
Final blend	New-model weights {.1,.25,.5,.75,1}	Learned model .75; baseline .25

Twelve linear and four learned-interaction fits produce one winner per family. The encoder and these three winners are then refitted on all training data: 19 supervised fits plus four preprocessing fits in total. All three family choices are frozen before loading official validation; only 15 fixed nonzero fusion settings are compared there. The first tied maximum retains the earlier model. Recorded full search time is 244.36 seconds locally. No test input or reference appears in the training/search code path.

5. Accuracy, ablation and paired error analysis

Method	Validation / hard (%)	Test accuracy (%)
Majority of valid numeric answers	86.22 / 69.01	69.16 (323/467)
Independent verifier (v1)	92.36 / 82.82	79.66 (372/467)
Pairwise baseline (v2)	93.86 / 86.20	81.37 (380/467)
Final question-conditioned fusion (v14)	94.36 / 87.32	83.08 (388/467)

5.1 Which interaction variant helped?

Frozen v14 family	Internal (%)	Standalone val. (%)	Best v2 blend (%)
Additive text control	92.22	93.73	93.86
Fixed product/distance features	91.69	93.73	93.86
Learnable low-rank conditioning	91.48	93.61	94.36

Adding fixed text interactions does not improve the best additive family on official validation. The learned model is also not the strongest standalone scorer. Its benefit appears after fusion: at new-model weights .1/.25/.5/.75/1, validation accuracy is 93.73/93.48/94.11/94.36/93.61%. Thus the measured improvement is best explained by complementary errors, not a blanket claim that a larger or more semantic model is always better.

The control families share the encoding and search ranges, but their selected dimensions, regularization or optimization differ. The linear internal grid supports matched comparisons at identical settings. The final nonlinear fusion comparison does not isolate question conditioning from all changes in objective, capacity and regularization. This limits causal claims about why the score improves.

5.2 Repairs, regressions and uncertainty

Comparison with v2	Repairs	Regressions	Net gain
Validation: 798 questions	10	6	+4 (+0.501 pp)
Hard validation: 355 questions	10	6	+4 (+1.127 pp)
Test: 467 questions	22	14	+8 (+1.713 pp)

A paired question bootstrap with 5,000 resamples and seed 7506 yields a 95% interval of [-0.50,+1.50] percentage points for overall validation and [-0.86,+4.28] for test. Exact McNemar p-values are .4545 and .2430, respectively. Both intervals include zero. The system improves on this particular bank, but the sample does not establish a statistically robust population improvement. Validation estimates are additionally optimistic because they selected the method.

The validation/test gap is partly structural: 443 validation questions have only one distinct valid answer, whereas the test bank contains disagreements throughout. Hard validation accuracy of 87.32% is more relevant than 94.36% overall when interpreting the intended difficulty. It still does not guarantee the test result.

6. Reliability and remaining failure modes

6.1 Confidence is informative but not calibrated

The confidence diagnostic is the maximum final ranking score for the selected answer. On validation, chosen-answer Brier score is .0831 and five-bin expected calibration error is .1300. On hard validation the corresponding values are .1604 and .1680. These are raw, post-selection diagnostics; no calibration model was fitted and the score must not be interpreted directly as a correctness probability.

Keep highest-scoring fraction	All validation: n / accuracy	Hard validation: n / accuracy
25%	200: 99.00%	89: 97.75%
50%	399: 99.25%	178: 94.94%
75%	599: 98.33%	267: 91.01%
100%	798: 94.36%	355: 87.32%

Higher scores usually identify a more reliable subset. At 50% coverage, overall accuracy is 99.25%, but hard-subset accuracy is 94.94%. The easy groups therefore inflate the overall curve. Coverage is rounded up to a whole question. These are descriptive development-set results, not independent test guarantees or an abstention policy: the submitted selector always returns an answer for every question.

Calibration is imperfect even in the highest bin: 549 validation selections have raw scores in [.8,1], mean score .9279 and empirical accuracy .9872. Lower bins are small and noisy. This pattern shows that a useful ordering can coexist with inaccurate probability scale. Temperature adjustment alone cannot improve an unchanged argmax ordering.

6.2 Qualitative examples from frozen validation errors

Average-score question (validation index 17). Three quizzes average 94 and the third score is 92; the first two average (3*94-92)/2=95. The selected trace instead inserts another known score of 92, solves an invented equation and returns 98. Local arithmetic can be consistent after the incorrect assumption. The failure is condition grounding, not multiplication or formatting.

Pairing question (index 44). There are 46 male and 48 female students. The selected solution correctly totals 94 students and 46 pairs, but computes 94-46=48 unpaired students. Each pair covers two students, so the correct result is 94-2*46=2. A numerical equality checker accepts 94-46=48; it does not know that the units are students versus pairs.

Ambiguous wording (index 32). After 20-12=8 paperclips remain, the phrase "1/4 more ... plus 7" is interpreted by the selected trace as 8+8/4+7=17, while the fixed reference is 9. This example warns against equating benchmark labels with unambiguous semantic truth. No reference, parser or scoring rule was changed; accuracy is always measured against the course specification.

These cases were examined after freezing for interpretation only. They did not produce ID-specific rules. More broadly, equation consistency is not a proof of the full derivation, and correct final answers can occur despite flawed intermediate reasoning [5].

7. Runtime, assets and reproducibility

7.1 Measured complete CPU evaluation

Item	Recorded value
Final test accuracy	388/467 = 83.08351177730192%
Development fresh prediction + scoring	13.48 s; four threads
Clean-folder fresh prediction + scoring	13.28 s; four threads
Hardware / platform	AMD Ryzen 7 5800H / Windows 11
Python	3.12.14
Total additional inference assets	6,246,341 bytes = 5.957 MiB
Course limits	30 s on reference CPU; 16 MiB assets

Both timings include interpreter startup, imports, data/asset loading, text processing, every model, and a separate official scoring process. OMP, OpenBLAS, MKL and NumExpr are capped at four threads. Local measurements are below the limit; eligibility on the reference Xeon Platinum 8457C remains subject to course reproduction. Course-supplied data and ordinary source are excluded from the additional-asset count [1].

Checkpoint asset	Uncompressed bytes
text_encoder.joblib	5,463,666
text_model.npz	199,670
config.json	132
v2/pair.npz	14,184
v2/neural.npz	567,980
v2/config.json	709

7.2 Self-contained executable artifact

The submission includes only the final inference path, original tools/data, fixed dependencies, selected-training and final-search code, small verification tests and necessary evidence. It omits obsolete models, discarded trial weights, IDE state, virtual environments, cached predictions and old reports. common.py, predict.py, score_predictions.py and check_data.py remain byte-identical to the supplied versions. The entry point is selector.py; all six checkpoint files are required.

```
python evaluate.py --split test --method selector.py
--checkpoint checkpoint --output-dir results
```

readme.md provides Windows/Linux and PyCharm setup. requirements.txt pins NumPy 2.1.3, SciPy 1.15.3, scikit-learn 1.6.1, joblib 1.4.2 and threadpoolctl 3.5.0. Prediction does not require training, internet, a GPU or an API key. verify.py checks asset hashes/size, new IDs, empty banks, batching, integer outputs and candidate-permutation scores. Test scripts cover gradient and parser boundaries.

Fixed-recipe train.py reads training data only and writes a separate checkpoint. In 90.74 seconds locally, it reproduces all learned model arrays, vocabulary, IDF and SVD values. Five asset files are byte-identical; joblib serialization metadata can differ although learned encoder contents match exactly. search.py replays the final 19-fit development search without reading test. MANIFEST.json identifies the standalone source/data/report snapshot; evidence/ stores hashes, freeze, tuning, official scores, paired analysis and reproduction records.

8. Discussion and conclusion

The strongest result is a modest, reproducible ensemble gain under a practical offline CPU budget. Relative ranking improves substantially over the initial majority baseline, but several increasingly complex alternatives fail to transfer their internal improvements to official validation. Mathematical consistency features improve a matched linear control yet only tie the established ensemble. Explicit question text is similarly insufficient on its own; the learned low-rank scorer contributes through complementary errors after fusion.

The final system remains limited by small training size, imperfect numeric/linguistic parsing, ambiguous benchmark wording, correlated candidate mistakes and distribution differences between easy and disagreement questions. Fixed representations constrain what can be recovered. Repeated adaptive validation use can overstate future gains. The earlier v1/v2 test scores were known before v14 development, although no test result selected the new checkpoint or triggered post-test tuning. The paired confidence interval includes zero, and no claim of robust population improvement is made.

The report distinguishes evidence from implementation checks: hash and permutation tests establish artifact/contract consistency; finite differences validate derivatives; the held-out comparisons assess selection; and the paired analysis quantifies sampling uncertainty. None proves that the model understands every mathematical condition. Future work would require genuinely independent questions to evaluate generalization, together with better grounding of quantities and units. The present submission preserves the frozen course protocol.

References

[1] DASE7506. Mini-project 2: Selecting Reliable LLM Reasoning. Supplied project guide and evaluation tools. https://github.com/xudongwu-0/DASE7506-MP2/blob/main/guide/GUIDE.md

[2] DASE7506. Supplied Lectures 4-7. Lecture 4, pp. 12-18 (task heads); Lecture 6, pp. 50-51,61 (pairwise preference and finite margins); Lecture 7, pp. 26,39,61 (group-relative signal and calibration). Course materials; not redistributed.

[3] K. Cobbe et al. Training Verifiers to Solve Math Word Problems. 2021. arXiv:2110.14168. https://arxiv.org/abs/2110.14168

[4] Budget-aware Test-time Scaling via Discriminative Verification. 2025. arXiv:2510.14913. https://arxiv.org/abs/2510.14913

[5] C. Zheng et al. ProcessBench: Identifying Process Errors in Mathematical Reasoning. 2024; ACL 2025 version. arXiv:2412.06559. https://arxiv.org/abs/2412.06559

[6] scikit-learn 1.6 documentation. Algorithms trained from scratch; NumPy/SciPy implement numerical operations. https://scikit-learn.org/1.6/user_guide.html

Data attribution: GSM8K under its retained MIT license; the course prepared candidate texts and fixed representations with Qwen3-0.6B at the revision specified in data/manifest.json. Keep data/LICENSE.txt and data/NOTICE.txt.

AI assistance declaration

OpenAI Codex assisted method and literature research, coding and experimentation, and report drafting and language editing.